

Basic data types in python

Introduction to Programming in Python

Rae Harbird, r.harbird@ucl.ac.uk

Variables and Constants

Numbers

Strings

Input and Output

Variables and Constants

Variables

- ▶ A variable is a named storage location in a computer program
- ▶ You 'define' a variable by telling the interpreter:
 - ▶ What name you will use to refer to it
 - ▶ The initial value of the variable
 - ▶ You use an assignment statement to place a value into a variable:

```
year_now = int(input("Enter the current year and press RETURN: "))
```

Variable names

- ▶ Variable names should describe the purpose of the variable
`canVolume` or `can_volume` is better than `cv`
- ▶ Use these simple rules:
 - ▶ Variable names must start with a letter or the underscore (`_`) character
 - ▶ Continue with letters (upper or lower case), digits or the underscore
 - ▶ Do not use other symbols (`?` or `%...`) and spaces are not permitted
 - ▶ Separate words with 'camelCase' notation, use upper case letters to signify word boundaries, or use `_`.
 - ▶ Don't use 'reserved' Python words.

Constants

- ▶ In Python a constant is a variable whose value should not be changed after it's assigned an initial value.
- ▶ It is a good practice to use all caps when naming constants:
- ▶ It is good style to use named constants to explain numerical values to be used in calculations:

```
age_now = int(input("Enter your current age in years: "))
```

```
if age_now > MAXIMUM_AGE:  
    print("Invalid age")
```

- ▶ Python will let you change the value of a constant *but* you should not do it.

Numbers

Number types

- ▶ In Python, there are several different types of numbers:
 - ▶ An integer value is a whole number without a fractional part. In Python, this type is called `int`.
 - ▶ When a fractional part is required (such as in the number 0.355), we use floating-point numbers, which are called `float` in Python.

Arithmetic

- ▶ Python supports all of the basic arithmetic operations:
 - ▶ +, - add, subtract
 - ▶ *, / ** multiply, divide, power
- ▶ In python, expressions are written a bit differently:
 - ▶ $\frac{a+b}{2}$ would be written as (a + b) / 2
- ▶ Precedence is similar to algebra:
 - ▶ PEMDAS - Parentheses, Exponent, Multiply/Divide, Add/Subtract

Mixing numeric types

- ▶ If you mix integer and floating-point values in an arithmetic expression, the result is a floating-point value:

`7 + 4.0 # Yields the floating value 11.0`

Arithmetic assignment operators

Python provides a shorthand notation for simple arithmetic operations followed by assignment, as in the following examples:

Example	Equivalent to
<code>number += extra</code>	<code>number = number + extra</code>
<code>total -= discount</code>	<code>total = total - discount</code>
<code>bonus *= 2</code>	<code>bonus = bonus * 2</code>
<code>time /= rush_factor</code>	<code>time = time / rush_factor</code>
<code>change %= 100</code>	<code>change = change % 100</code>
<code>amount *= n1 + n2</code>	<code>amount = amount * (n1 + n2)</code>

This shorthand should not be used at the expense of having a program that is easily readable.

Floor Division

- ▶ When you divide two integers with the `/` operator, you get a floating-point value so `7 / 4` yields `1.75`
- ▶ We can also perform *floor division* using the `//` operator. The `//` operator computes the quotient and discards the fractional part:

`7 // 4`

Evaluates to 1 because 7 divided by 4 is 1.75 with a fractional part of 0.75, which is discarded

Remainder (or Modulo Divide)

- ▶ If you are interested in the remainder of dividing two integers, use the % operator (called modulus):

```
remainder = 7 % 4
```

- ▶ The value of remainder will be 3, this is sometimes called modulo divide.

Using built-in functions

- ▶ A function is a collection of programming instructions that carry out a particular task.
- ▶ The `print()` function can display information, but there are many other functions available in Python.
- ▶ Most functions return a value. That is, when the function completes its task, it passes a value back to the point where the function was called. For example: the call `abs(-173)` returns the value `173`.
- ▶ The value returned by a function can be stored in a variable:

```
distance = abs(x)
```

- ▶ You can use a function call as an argument to the `print()` function

```
print(abs(-173))
```

Built-in mathematical functions

[Link here](#) for a list of all python built-in functions.

Function	Returns
<code>abs(x)</code>	The absolute value of x
<code>round(x)</code> <code>round(x, n)</code>	The floating point value of x rounded to a whole number or to n decimal places.
<code>max(x1, x2, x3, ...,xn)</code>	The largest value from the parameters.
<code>min(x1, x2, x3, ...,xn)</code>	The smallest value from the parameters.

Rounding errors

- ▶ Floating point values are not exact. This is a limitation of binary values; not all floating point numbers have an exact representation. Let's try this:

```
price = 4.35
quantity = 100
total = price * quantity
# Should be 100 * 4.35 = 435.00
print(total)
```

- ▶ This doesn't quite do what you might expect. That's OK, we can:
 - ▶ round to the nearest integer using the `round()` function.
 - ▶ or display a fixed number of digits after the decimal separator. We will see how to do that in the section on Input and Output.

Strings

Definition

Start with a simple definition:

- ▶ A string is a sequence of characters consisting of characters - letters, numbers, punctuation marks, spaces ...
- ▶ In Python, string literals are specified by enclosing a sequence of characters within a *matching pair* of either single or double quotes:

```
print("This is a string.", 'So is this.')
```

- ▶ By allowing both types of delimiters, Python makes it easy to include an apostrophe or quotation mark within a string:

```
message = 'He said "Hello"'
```

Remember to use matching pairs of quotes, single with single, double with double

String length

- ▶ The number of characters in a string is called the length of the string (For example, the length of “Harry” is 5)
- ▶ You can compute the length of a string using Python’s `len()` function:

```
length = len("World!") # length is 6
```

- ▶ A string of length 0 is called the empty string. It contains no characters and is written as `""` or `''`

Concatenation and Repetition

- ▶ You can 'add' one string onto the end of another:

```
firstName = "Harry"  
lastName = "Morgan"  
name = firstName + lastName # HarryMorgan  
print("my name is:", name)
```

- ▶ Using "+" to concatenate strings is an example of a concept called operator overloading. The + operator performs different functions of variables of different types.

- ▶ The * operator is also overloaded

```
dashes = "-" * 50
```

results in the string

```
"-----"
```

Covert numbers to Strings

- ▶ You will find out very quickly that you can't use the `print()` to print a number.
- ▶ You must use the `str()` function to convert between numbers and strings:

```
balance = 888.88
dollars = 888
balanceAsString = str(balance)
dollarsAsString = str(dollars)
print(balanceAsString)
print(dollarsAsString)
```

Escape sequences and special characters

- ▶ Sometimes you might want to print a double quote but how can you do that?

- ▶ Preface the " with a "\" inside the double quoted string

```
print("He said \"Hello\"")
```

- ▶ To print a backslash, preface the \ with another \

```
print("C:\\Temp\\Secret.txt")
```

Input and Output

Input

- ▶ You can use the `input()` function to prompt the user for a value.

```
name = input("Enter your name: ")
```

- ▶ If you need a numeric value then you can *wrap* the `input` function in the `int()` function.

```
age_now = int(input("Enter your current age in years: "))
```

- ▶ Similarly, there is also a `float()` function:

```
temperature_now = float(input("Enter the temperature: "))
```

Formatted Output

- ▶ Outputting floating point values can look strange:

Price per litre: 1.21997

- ▶ There are two ways to control the output appearance of printed output:
 - ▶ use format specifiers
 - ▶ use the string `format()` method
- ▶ These are both a bit tricky for new programmers to understand.
 - ▶ FWIW, my advice is: choose one style and stick to it.

Using format specifiers

► Example 1:

```
price = 1.2124  
print("Price per litre %.2f" %(price))
```

Output:

```
Price per litre: 1.21
```

► Example 2 - specify the size of the field:

```
print("Price per litre: %10.2f" %(price))
```

Output:

```
Price per litre:      1.22
```

Using the string format() method

- ▶ Alternatively, you can use the `format()` method. See here for an explanation.

- ▶ Example:

```
price = 10.345
quantity = 5
total_cost = price * quantity
print("Quantity: {0:2d} unit price: {1:.4f} TOTAL: {2:.2f}"
      .format(quantity, price, total_cost))
```

Output:

```
Quantity:  5 unit price: 10.3450 TOTAL: 51.73
```

- ▶ Placeholders `{}` are used to mark where the output should go.